# Nano-Swarm — parallel cheap retrieval with bounded fan-out

SPEC-TH-010 v0.1 DRAFT Jeremie Nunez

## 1 Context

The Explorer subsystem of Thalamus must gather evidence from the open web on every research cycle. A single call to a mid-tier model with web search returns one perspective and 1–2 domains, which is too narrow and too monolithic. The Nano-Swarm replaces that single call with a fan-out of many cheap nano-model workers, each constrained to a distinct *lens* (angle, specialty, or sub-topic). Fan-out is bounded, executed in fixed-size waves, and every worker is independent, so a partial failure never blocks the cycle.

## 2 Goals

- Turn a small set of high-level exploration queries into a bounded set of micro-queries, each paired with a distinct retrieval lens.
- Execute the micro-queries in parallel waves with a fixed concurrency cap and an inter-wave delay to respect upstream rate limits.
- Produce a deduplicated set of retrieval artifacts (one per unique source URL, plus synthetic artifacts when a worker returns text without citations).
- Emit deterministic swarm statistics (call counts, success ratio, unique domains, wall time, estimated cost) on every cycle.
- Fail open: any single worker failure is absorbed; the cycle returns whatever succeeded.

## 3 Non-Goals

**Non-Goal**

This spec does not cover: (a) generation of the upstream exploration queries (owned by the Scout), (b) relevance or novelty scoring (owned by the Curator, SPEC-TH-012), (c) persistence of artifacts to the feed or knowledge store, (d) the typed external source adapters (SPEC-TH-011), and (e) the choice of underlying nano model or its pricing.

## 4 Invariants

**Invariant**

Fan-out is bounded: the number of dispatched micro-queries per cycle never exceeds `MAX_MICRO_QUERIES`, regardless of the number of upstream queries or lenses.

**Invariant**

Concurrency is bounded: at most `WAVE_SIZE` nano calls are in-flight at any moment; successive waves are separated by at least `WAVE_DELAY_MS` of wall time (except after the last wave).

**Invariant**

Every single nano call is bounded in time by `CALL_TIMEOUT_MS` and every error, timeout, or non-2xx response is converted into a typed failure result — never a thrown exception that escapes the swarm.

**Invariant**

Artifact URLs are unique across the returned set after normalization (lowercased host, stripped tracking params, no trailing slash).

**Invariant**

If no upstream credentials are configured, the swarm returns an empty artifact set and well-formed stats; it never crashes the caller.

## 5 Interface

Public surface of the swarm. The consumer is the Explorer orchestrator.

```
export interface ExplorationQuery {
  query: string;
  type: "web" | "academic" | "market";
  signal: string;
  priority: number;
  maxDepth: number;
}

export interface RetrievalArtifact {
  url: string;              // unique source URL, or "nano://<workerId>"
      synthetic
  title: string;
  body: string;             // <= 3000 chars, markdown-stripped
  entities: ExtractedEntities;
  dataPoints: string[];
  sourceQuery: string;      // upstream query that produced this
     artifact
  depth: 0;
}

export interface SwarmStats {
  totalCalls: number;
  successCalls: number;
  failedCalls: number;
  wallTimeMs: number;
  uniqueDomains: number;
  totalUrls: number;
  totalChars: number;
  estimatedCost: number;
}

export class NanoSwarm {
  crawl(upstream: ExplorationQuery[]): Promise<{
    articles: RetrievalArtifact[];
    urlsCrawled: number;
    stats: SwarmStats;
  }>;
}
```

## 6 Scenarios

**Scenario: Nominal fan-out**

**Given** a list of 6 upstream exploration queries with mixed `type` values
**When** `NanoSwarm.crawl` is invoked
**Then** the swarm decomposes them into exactly `MAX_MICRO_QUERIES` micro-queries, each tagged with a distinct lens identifier
**And** the micro-queries are dispatched in waves of `WAVE_SIZE`
**And** the returned `articles` contain no duplicate normalized URLs
**And** `stats.totalCalls` equals the number of micro-queries dispatched

**Scenario: Partial failure absorption**

**Given** a cycle where 30% of nano calls return HTTP errors or time out
**When** the swarm completes
**Then** the failed calls are counted in `stats.failedCalls`
**And** successful calls still contribute artifacts to the output
**And** no exception is raised to the caller

**Scenario: Credential absence**

**Given** the upstream API key is not configured in the environment
**When** the swarm is invoked
**Then** every nano call returns a typed failure result with an explicit error marker
**And** `articles` is empty
**And** `stats.successCalls` is 0 and `stats.totalCalls` equals the number of dispatched micro-queries

**Scenario: Rate-limit backpressure**

**Given** a set of 50 micro-queries
**When** the swarm executes them
**Then** the total wall time is at least $(\lceil 50/\texttt{WAVE_SIZE} \rceil - 1) \times \texttt{WAVE_DELAY_MS}$ milliseconds
**And** at no point are more than `WAVE_SIZE` requests in flight concurrently

**Scenario: Synthetic artifact fallback**

**Given** a worker that returns substantive text but cites no URLs
**When** results are merged
**Then** exactly one synthetic artifact is produced with `url = "nano://<workerId>"`
**And** that artifact carries the worker's `sourceQuery` and extracted entities

## 7 Acceptance Criteria

**Acceptance Criterion AC-1**

Given an empty upstream query list, `crawl` returns `articles: []`, `urlsCrawled: 0`, and `stats.totalCalls: 0`.

**Acceptance Criterion AC-2**

The number of dispatched nano calls in a single cycle is always $\leq$ `MAX_MICRO_QUERIES`, verified by intercepting the HTTP transport.

**Acceptance Criterion AC-3**

Concurrency never exceeds `WAVE_SIZE`: a test that instruments the nano caller records at no tick a live-call count greater than `WAVE_SIZE`.

**Acceptance Criterion AC-4**

After normalization, no two artifacts in a single output share the same URL.

**Acceptance Criterion AC-5**

A 4xx or 5xx HTTP response from a single worker increments `stats.failedCalls` by 1 and does not reduce the contribution of other workers.

**Acceptance Criterion AC-6**

A worker whose text exceeds 200 characters and has 0 extracted URLs produces exactly one synthetic artifact with a `nano://` URL.

**Acceptance Criterion AC-7**

`stats.wallTimeMs` is populated on every return and is monotonically greater than the elapsed time measured outside the call by no more than 50 ms.

## 8 Traceability

| AC | Test file | Test name |
|---|---|---|
| AC-1 | `packages/thalamus/tests/nano-swarm.empty.spec.ts` | `returns empty on empty input` |
| AC-2 | `packages/thalamus/tests/nano-swarm.fanout.spec.ts` | `caps fan-out at MAX_MICRO_QUER` |
| AC-3 | `packages/thalamus/tests/nano-swarm.concurrency.spec.ts` | `never exceeds wave size` |
| AC-4 | `packages/thalamus/tests/nano-swarm.dedup.spec.ts` | `deduplicates URLs after normal` |
| AC-5 | `packages/thalamus/tests/nano-swarm.failure.spec.ts` | `absorbs per-worker failures` |
| AC-6 | `packages/thalamus/tests/nano-swarm.synthetic.spec.ts` | `produces synthetic artifact wh` |
| AC-7 | `packages/thalamus/tests/nano-swarm.stats.spec.ts` | `reports accurate wall time` |

## 9 Open Questions

- Should `WAVE_SIZE`, `WAVE_DELAY_MS`, and `MAX_MICRO_QUERIES` be tunable per-cycle (config) rather than compile-time constants?
- Should the swarm expose per-wave progress as an async iterator for the UI layer, or is the final batch result sufficient?
- Is cost estimation in `stats` authoritative, or should it be computed downstream from a billing feed?